

CANDIDATE ANSWER

Options: (Expected answer indicated with a tick)

- ☒ ☐ I love programming
☐ Just doing it to pass this course

No Comments

QUESTION 2



Correct Answer

Score 10

Plus and Equals To > Coding Python 3 CS101

QUESTION DESCRIPTION

Challenge

Write a function named `print_plus_and_equals` that takes one argument `size` of type `int`, and returns a string showing a plus sign followed by an equals sign with the specified size.

Sample

```
>>> print_plus_and_equals(6)
**
**
*****
*****
**
**

*****

*****
```

```
>>> print_plus_and_equals(4)
**
**
****
****
**
**

****

****
```

Constraints

- `size` is an even integer - not less than 4.

CANDIDATE ANSWER

Language used: Python 3

```
1 # Enter your code here. Read input from STDIN.
2
3 #input size of plus
```

```

4 size = int(input())
5
6
7 def print_plus_and_equals(size):
8     speck = size // 2 #initialize the variable 'speck' to be half of 'size'
9
10    #initialize an empty string to store the result
11    result = ''
12
13    for i in range(2): #loop to append two lines of 2 '*' with leading spaces
14 to result
15        result += (speck - 1) * ' ' + 2 * '*' + '\n'
16    for j in range(2): #loop to append two lines of 'size' '*' to result to
17 create a block
18        result += size * '*' + '\n'
19    for k in range(2): #loop to append two lines of 2 '*' with leading spaces
20 to result
21        result += (speck - 1) * ' ' + 2 * '*' + '\n'
22
23    result += '\n' * speck + size * '*' + '\n' * 3 + size * '*'
24
25    #return the final result string
26    return result

```

TESTCASE	DIFFICULTY	TYPE	STATUS	SCORE	TIME TAKEN	MEMORY USED
Testcase 0	Easy	Sample case	✔ Success	3	0.0143 sec	9.65 KB
Testcase 1	Easy	Sample case	✔ Success	3	0.016 sec	9.71 KB
Testcase 2	Easy	Hidden case	✔ Success	4	0.0141 sec	9.54 KB

No Comments

QUESTION 3



Correct Answer

Score 10

Heart Rate Measurements > Coding

QUESTION DESCRIPTION

Sample

The patients' Heart-Rate (HR) measurements were taken over a number of tests/situations - resting, walking slowly, running on treadmill, after minute recovery. We would like to print the report showing the heart rate of each patient and the average of the measurements for each test conducted across.

For example, suppose we have the following patients heart-rate records:

Patient ID	Resting HR	Walking Slowly HR	Running on Treadmill HR	After Minute Recovery HR
P001	72	91	130	120
P002	75	90	135	118
P003	71	94	139	110
P004	73	93	142	105

Patients Heart-Rate records are represented as a nested list in Python:

```
Patients_HR = [ ["P001", 72, 91, 130, 120], ["P002", 75, 90, 135, 118],  
["P003", 71, 94, 139, 110], ["P004", 73, 93, 142, 105] ]
```

Write a function **HR_tests_reports** that takes **Patients_HR** list as a parameter and print the report showing the heart rate of each patient and the average of the measurements for each test conducted across.

```
>> Patients_HR = [ ["P001", 72, 91, 130, 120], ["P002", 75, 90, 135, 118],  
["P003", 71, 94, 139, 110], ["P004", 73, 93, 142, 105] ]  
  
>> HR_tests_reports(Patients_HR)  
Average Heart Rate of Each Patient:  
P001: 413  
P002: 418  
P003: 414  
P004: 413  
  
Average Heart Rate Tests Results:  
Resting: 72.75  
Walking Slowly: 92  
Running on Treadmill: 136.5  
After Minute Recovery: 113.25
```

INTERVIEWER GUIDELINES

CANDIDATE ANSWER

Language used: Python 3

```
1  
2 def HR_tests_reports(Patients_HR):  
3  
4     #initialize average variables for resting, walking, treadmill, and  
5     recovery  
6     avg_resting, avg_walking, avg_treadmill, avg_recovery = 0, 0, 0, 0  
7  
8     #get the total number of patients  
9     patients = len(Patients_HR)  
10  
11     #print the average Heart Rate of each patient  
12     print("Average Heart Rate of Each Patient:")  
13     for i in Patients_HR:  
14         print(str(i[0]) + ': ' + str((sum(i[1:]))))  
15  
16     #accumulate individual test values for later average calculation  
17     avg_resting += i[1]  
18     avg_walking += i[2]  
19     avg_treadmill += i[3]  
20     avg_recovery += i[4]  
21     print()  
22  
23     #print the average Heart Rate test results for all patients  
24     print("Average Heart Rate Tests Results:")  
25     print("Resting:", avg_resting / patients)  
26     print("Walking Slowly:", avg_walking / patients)  
27     print("Running on Treadmill:", avg_treadmill / patients)
```

```
28 print("After Minute Recovery:", avg_recovery / patients)
```

TESTCASE	DIFFICULTY	TYPE	STATUS	SCORE	TIME TAKEN	MEMORY USED
Testcase 0	Easy	Sample case	✔ Success	1	0.0187 sec	10.2 KB
Testcase 1	Easy	Hidden case	✔ Success	2	0.0182 sec	10.3 KB
Testcase 2	Easy	Sample case	✔ Success	1	0.0812 sec	10 KB
Testcase 3	Easy	Hidden case	✔ Success	2	0.0166 sec	10.3 KB
Testcase 4	Easy	Hidden case	✔ Success	2	0.0209 sec	10.3 KB
Testcase 5	Easy	Hidden case	✔ Success	2	0.0196 sec	10.2 KB

No Comments

QUESTION 4



Correct Answer

Score 10

Chickenville > Coding CS101 Python 3

QUESTION DESCRIPTION

Background

In the online game of *Chickenville*, a farmer must raise a certain number of chickens to pass a given *level*. Difficulty of the game is determined by three values: *start*, *multiplier*, *bonus*.

See the sample cases below to illustrate the mechanics of the game.

Task

Given the values for *start*, *multiplier*, and *bonus*, write a program to calculate the number of chicken that the farmer must raise to pass a *level*.

Function Description

To implement the given task, write the following function:

1. ***level_up*** that takes four argument: ***start***, ***multiplier***, ***bonus***, ***level***. The function should **return** the number of chickens.

Constraints

- You must use recursion to calculate the number of chickens—you may not use for or while loops, or any other form of iteration.
- You may not reference global variables.
- You may not use any helper functions.
- Input and output will be handled by HackerRank—you should not read input or display values yourself.
- *start* and *bonus* are non-negative integers; *multiplier* and *level* are positive integers.

▼ Input Format For Custom Testing

The first line contains *start*, the number of chickens with which the farmer begins the game.
The second line contains *multiplier*, the factor with which to multiply the number of chickens from the previous level.
The third line contains *bonus*, the number to add to the number of chickens from the previous level.
The fourth line contains *level*, which is the desired level for which the number of chickens must be calculated.

▼ Sample Case 0

Sample Input For Custom Testing

2
3
5
6

Sample Output

1091

Explanation

The farmer begins the game with two chickens ($start = 2$). To get to the next level (level 2), she must raise $2 \times multiplier + bonus = 2 \times 3 + 5 = 11$ chickens. For level 3, she must raise $11 \times 3 + 5 = 38$ chickens. For level 4, $38 \times 3 + 5 = 119$ chickens. For level 5, $119 \times 3 + 5 = 362$ chickens. For the final level, $level = 6$, $362 \times 3 + 5 = 1091$ chickens.

▼ Sample Case 1

Sample Input For Custom Testing

1
2
0
10

Sample Output

512

Explanation

The farmer begins the game with one chickens ($start = 1$). To get to the next level (level 2), she must raise $1 \times multiplier + bonus = 1 \times 2 + 0 = 2$ chickens. For level 3, she must raise $2 \times 2 + 0 = 4$ chickens. For level 4, $4 \times 2 + 0 = 8$ chickens. For level 5, $8 \times 2 + 0 = 16$ chickens. For level 6, $16 \times 2 + 0 = 32$ chickens. For level 7, $32 \times 2 + 0 = 64$ chickens. For level 8, $64 \times 2 + 0 = 128$ chickens. For level 9, $128 \times 2 + 0 = 256$ chickens. For the final level, $level = 10$, $256 \times 2 + 0 = 512$ chickens.

INTERVIEWER GUIDELINES

CANDIDATE ANSWER

Language used: Python 3

```
1 def level_up(start, multiplier, bonus, level):  
2     #if level is equal to 1, return the current value of 'start'  
3     if level == 1:  
4         return start  
5     #else, recursively call level_up function with updated values for 'start'  
6     based on multiplier, bonus, and reduced level  
7     else:  
8         return level_up(((start * multiplier) + bonus), multiplier, bonus,  
9                           level - 1)
```

TESTCASE	DIFFICULTY	TYPE	STATUS	SCORE	TIME TAKEN	MEMORY USED
TestCase 0	Easy	Sample case	✔ Success	1	0.1492 sec	11.7 KB
TestCase 1	Easy	Sample case	✔ Success	1	0.0765 sec	11.6 KB

TestCase 2	Easy	Sample case	✔ Success	1	0.0322 sec	11.7 KB
TestCase 3	Easy	Sample case	✔ Success	1	0.1056 sec	11.8 KB
TestCase 4	Easy	Sample case	✔ Success	1	0.0263 sec	11.8 KB
TestCase 5	Easy	Hidden case	✔ Success	1	0.0265 sec	11.7 KB
TestCase 6	Easy	Hidden case	✔ Success	1	0.0297 sec	11.7 KB
TestCase 7	Easy	Hidden case	✔ Success	1	0.047 sec	11.4 KB
TestCase 8	Easy	Hidden case	✔ Success	1	0.0742 sec	11.7 KB
TestCase 9	Easy	Hidden case	✔ Success	1	0.0389 sec	11.7 KB

No Comments

QUESTION 5



Correct Answer

Score 10

Partition modulo N > Coding

QUESTION DESCRIPTION

Challenge

Generalize `partition_modulo_three` by writing a function `partition_modulo_n` that accepts `n` and `nums`, and partitions `nums` according to the $(\text{mod } n)$ relation. Every possible remainder, from 0 to `n - 1`, is a key in the returned `dict`. Its value is a list containing elements of `nums` that are equivalent to it $(\text{mod } n)$.

The function must not modify `nums`, i.e., it has no side-effects.

Sample

```
>>> partition_modulo_n(3, [0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11])
{0: [0, 3, 6, 9], 1: [1, 4, 7, 10], 2: [2, 5, 8, 11]}
>>> partition_modulo_n(3, [-6, -5, -4, -3, -2, -1, 0, 1, 2, 3, 4, 5])
{0: [-6, -3, 0, 3], 1: [-5, -2, 1, 4], 2: [-4, -1, 2, 5]}
>>> partition_modulo_n(3, [17, 20, -16, 34, 28, 47, -49, 37, -43, -19, -8, 19])
{0: [], 1: [34, 28, 37, -8, 19], 2: [17, 20, -16, 47, -49, -43, -19]}
>>> partition_modulo_n(4, [0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11])
{0: [0, 4, 8], 1: [1, 5, 9], 2: [2, 6, 10], 3: [3, 7, 11]}
>>> partition_modulo_n(4, [-6, -5, -4, -3, -2, -1, 0, 1, 2, 3, 4, 5])
{0: [-4, 0, 4], 1: [-3, 1, 5], 2: [-6, -2, 2], 3: [-5, -1, 3]}
>>> partition_modulo_n(4, [17, 20, -16, 34, 28, 47, -49, 37, -43, -19, -8, 19])
{0: [20, -16, 28, -8], 1: [17, 37, -43, -19], 2: [34], 3: [47, -49, 19]}
>>> partition_modulo_n(-4, [0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11])
{-3: [1, 5, 9], -2: [2, 6, 10], -1: [3, 7, 11], 0: [0, 4, 8]}
>>> partition_modulo_n(-4, [-6, -5, -4, -3, -2, -1, 0, 1, 2, 3, 4, 5])
{-3: [-3, 1, 5], -2: [-6, -2, 2], -1: [-5, -1, 3], 0: [-4, 0, 4]}
>>> partition_modulo_n(-4, [17, 20, -16, 34, 28, 47, -49, 37, -43, -19, -8, 19])
{-3: [17, 37, -43, -19], -2: [34], -1: [47, -49, 19], 0: [20, -16, 28, -8]}
```

Input/Output

The input consists of 2 lines. The first line contains `n`. The second line contains the elements of `nums` separated by space.

Constraints

- `isinstance(nums, list)`
- `isinstance(n, int)`
- `all(isinstance(i, int) for i in nums)`

INTERVIEWER GUIDELINES

CANDIDATE ANSWER

Language used: **Python 3**

```

1 # Enter your code here.
2 def partition_modulo_n(n,nums):
3     mod_d={}
4     if n >=0: #check if n is non-negative
5         for i in range(n):
6             mod_d[i] = [] #initiliaze an empty list for all remainders
7     else:
8         for i in range(n+1,1): #loop through the range from n+1 to 1
9 (exclusive) if n is negative
10         mod_d[i] = [] #initiliaze an empty list for all remainders
11
12     #iterate through the input numbers and group them by remainder
13     for i in nums:
14         mod_d[i%n].append(i)
15     return mod_d

```

TESTCASE	DIFFICULTY	TYPE	STATUS	SCORE	TIME TAKEN	MEMORY USED
Testcase 0	Easy	Sample case	✓ Success	0.6	0.0259 sec	9.79 KB
Testcase 1	Easy	Sample case	✓ Success	0.6	0.0208 sec	9.64 KB
Testcase 2	Easy	Sample case	✓ Success	0.6	0.0128 sec	9.45 KB
Testcase 3	Easy	Hidden case	✓ Success	0.75	0.0698 sec	9.62 KB
Testcase 4	Easy	Hidden case	✓ Success	0.75	0.053 sec	9.46 KB
Testcase 5	Easy	Sample case	✓ Success	0.6	0.0162 sec	9.63 KB
Testcase 6	Easy	Sample case	✓ Success	0.6	0.0134 sec	9.81 KB
Testcase 7	Easy	Sample case	✓ Success	0.6	0.0243 sec	9.52 KB
Testcase 8	Easy	Hidden case	✓ Success	0.75	0.0163 sec	9.64 KB
Testcase 9	Easy	Hidden case	✓ Success	0.75	0.0162 sec	9.39 KB
Testcase 10	Easy	Sample case	✓ Success	0.6	0.0169 sec	9.6 KB
Testcase 11	Easy	Sample case	✓ Success	0.6	0.0128 sec	9.64 KB
Testcase 12	Easy	Sample case	✓ Success	0.6	0.014 sec	9.63 KB
Testcase 13	Easy	Hidden case	✓ Success	0.8	0.0191 sec	9.54 KB
Testcase 14	Easy	Hidden case	✓ Success	0.8	0.0147 sec	9.46 KB

No Comments

QUESTION 6



Correct Answer

The RAT > Coding

Strings

CS101

QUESTION DESCRIPTION



Problem:

Disclaimer: This question in no way encourages the behavior portrayed.

You have been scorned by your friend and now you seek revenge. You now look in every single submission of the friend to check for plagiarism from a certain suspected text. You have decided to write a function `how_similar` that takes two strings, `toCheck` and `checkFrom` as parameters, checks for similarity in each character (revenge has you blinded) and returns the percentage similarity that `toCheck` has with `checkFrom`. Remember, in similarity checking, you see how much of the text **being checked** is copied.

The percentage is calculated by the value divided by the total and then multiplied with 100.

Sample:

```
>>>how_similar('hello','hello')
100.0
>>>how_similar('YOLO','Love is eternal')
25.0
>>>how_similar('bye man','nab yem')
100.0
```

Input:

The input has `toCheck` in one line followed by `checkFrom` in the next.

Constraints:

`isinstance(toCheck, str)` is `True` and `isinstance(checkFrom, str)` is `True`.

INTERVIEWER GUIDELINES

CANDIDATE ANSWER

Language used: **Python 3**

```
1 # Enter your code here.
2 #prompt the user to enter the strings for comparison
3 toCheck = input()
4 checkFrom = input()
5
6 def how_similar(toCheck,checkFrom ):
7     count = 0 #initialize a counter for common characters
8
9     for i in toCheck: #iterate through each character in the first string
10         if i in checkFrom: #check if the character is present in the second
11 string
12             count += 1 #increment the counter if the character is common to
13 both strings
```

```

14
15     #calculate the similarity percentage based on the count and length of the
16 first string
17     similarity_percentage = (count / len(toCheck)) * 100
18
19     #return the computed similarity
20     return similarity_percentage

```

TESTCASE	DIFFICULTY	TYPE	STATUS	SCORE	TIME TAKEN	MEMORY USED
Testcase 0	Easy	Sample case	✔ Success	2	0.0152 sec	9.37 KB
Testcase 1	Easy	Hidden case	✔ Success	2	0.0148 sec	9.42 KB
Testcase 2	Easy	Sample case	✔ Success	2	0.1021 sec	9.55 KB
Testcase 3	Easy	Hidden case	✔ Success	2	0.0129 sec	9.67 KB
Testcase 4	Easy	Hidden case	✔ Success	2	0.019 sec	9.37 KB

No Comments